PRAKTIKUM 5

RDD, Transformation, Action, dan Partition pada Apache Spark

Praktikum ini melanjutkan pengenalan Spark. Fokusnya adalah RDD sebagai abstraksi data terdistribusi, partition sebagai unit pemrosesan, transformation, action, lazy evaluation, dan WordCount.

```
HDFS → menyimpan data

Spark → memproses data

RDD → representasi dataset terdistribusi

Partition → bagian RDD yang diproses

Task → pekerjaan pada partition

Executor → menjalankan task
```

Catatan: Praktikum ini melanjutkan Spark Standalone dari Praktikum 4. Konsep RDD tetap sama jika Spark dijalankan menggunakan YARN.

1. Tujuan

- Memahami RDD.
- Membedakan HDFS Block dan RDD Partition.
- Membuat RDD dengan parallelize().
- Membaca data HDFS dengan textFile().
- Memahami transformation dan action.
- Memahami lazy evaluation.
- Menggunakan map(), filter(), flatMap(), reduce(), dan reduceByKey().
- Mengamati partition.
- Membuat WordCount menggunakan RDD.

2. RDD, HDFS Block, dan Partition

Tiga konsep berikut jangan tertukar:

Konsep	Fungsi	Lapisan
HDFS Block	Membagi data untuk penyimpanan	Storage
RDD Partition	Membagi data untuk pemrosesan	Processing
RDD	Abstraksi dataset terdistribusi	Spark

```
HDFS → Block → DataNode

       ↓ dibaca Spark

RDD → Partition → Task → Executor
```

4. Transformation dan Action

Jenis	Contoh	Keterangan
Transformation	map(), filter(), flatMap()	Membentuk hasil/RDD baru; lazy
Transformation	reduceByKey()	Menggabungkan value berdasarkan key
Action	collect(), count(), first()	Memicu eksekusi
Action	reduce()	Menghasilkan satu nilai

Pada praktikum ini, setiap operasi dipilih berdasarkan fungsi yang berbeda:

a. parallelize() membuat RDD dari data local
b. textFile() membaca data dari file/HDFS
c. map() mengubah setiap elemen
d. filter() menyaring elemen
e. flatMap() memecah satu elemen menjadi beberapa elemen
f. reduceByKey() melakukan agregasi berdasarkan key
g. count(), collect(), first(), dan reduce() mengambil hasil;
h. getNumPartitions() dan glom() membantu mengamati partition.

5. Lazy Evaluation

Spark tidak langsung menjalankan seluruh transformation ketika ditulis. Spark menunda eksekusi sampai ada action.

```
RDD → filter() → map() → count()

                    ↑

              action memicu eksekusi
```

Lazy bukan berarti data tidak diproses. Artinya eksekusi ditunda sampai hasil benar-benar diperlukan.

6. Persiapan

- Docker Desktop berjalan.
- Spark Master dan Worker dari Praktikum 4 tersedia.
- HDFS cluster tersedia.
- Windows menggunakan Command Prompt (CMD).
- Browser digunakan untuk Spark UI.

Buka CMD:

```
cd C:\bigdata\spark-praktikum

docker compose -f docker-compose.spark.yml up -d

docker ps
```

Pastikan spark-master, spark-worker-1, dan spark-worker-2 berstatus Up.

7. Menjalankan Spark Shell

```
docker exec -it spark-master bash

/opt/spark/bin/spark-shell --master spark://spark-master:7077
```

Catatan: Mulai langkah ini Anda berada di Linux dalam container. Perintah berikut diketik di spark-shell.

8. Membuat RDD Pertama

operasi parallelize(): digunakan untuk membuat RDD dari data yang sudah tersedia sebagai collection di program, misalnya Seq. Ini cocok untuk latihan dan pengujian data kecil sebelum membaca dataset dari file/HDFS.

Ilustrasi: Data lokal [10, 20, 30, 40, 50] → parallelize() → RDD → beberapa partition yang dapat diproses secara paralel.

```
val data = Seq(10, 20, 30, 40, 50)

val rdd = sc.parallelize(data)

rdd.collect()
```

Hasil:

```
Array(10, 20, 30, 40, 50)
```

parallelize() membuat RDD dari koleksi data yang tersedia pada program.

9. Melihat Jumlah Partition

getNumPartitions(): digunakan untuk mengetahui berapa banyak partition yang dimiliki sebuah RDD. Partition adalah unit data yang menjadi dasar pembagian pekerjaan Spark.

Guna glom(): digunakan untuk melihat elemen yang berada di masing-masing partition. Ini sangat membantu untuk memahami bagaimana data dibagi untuk pemrosesan paralel.

Ilustrasi: RDD [1,2,3,4,5,6,7,8] → 4 partition → P1, P2, P3, P4. Isi setiap partition dapat diamati dengan glom().

Catatan: jumlah partition tidak sama dengan jumlah DataNode dan tidak otomatis sama dengan jumlah executor.

```
rdd.getNumPartitions
```

Catat jumlah partition yang muncul.

Buat RDD dengan 4 partition:

```
val rdd4 = sc.parallelize(Seq(1,2,3,4,5,6,7,8), 4)

rdd4.getNumPartitions

rdd4.glom().collect()
```

glom() membantu melihat isi masing-masing partition.

Catatan: Jumlah partition tidak otomatis sama dengan jumlah DataNode.

10. Transformation: map()

map(): digunakan untuk mengubah setiap elemen RDD menjadi bentuk/nilai baru. Setiap satu input menghasilkan satu output.

Ilustrasi sebelum → sesudah:

[1, 2, 3, 4, 5] → map(x => x * 2) → [2, 4, 6, 8, 10]

Contoh penggunaan: mengalikan nilai, mengubah format data, mengambil atribut tertentu, atau menghitung nilai baru untuk setiap record.

Penting: map() adalah transformation, sehingga belum menjalankan pekerjaan ketika baris map() dibuat. Action seperti collect() memicu eksekusi.

```
val angka = sc.parallelize(Seq(1,2,3,4,5))

val kaliDua = angka.map(x => x * 2)
```

Belum ada hasil yang diminta. Sekarang jalankan action:

```
kaliDua.collect()
```

Hasil:

```
Array(2, 4, 6, 8, 10)
```

11. Transformation: filter()

filter(): digunakan untuk menyaring data. Hanya elemen yang memenuhi kondisi (bernilai true) yang dipertahankan.

Ilustrasi sebelum → kondisi → sesudah:

[1, 2, 3, 4, 5, 6] → filter(x % 2 == 0) → [2, 4, 6]

Contoh penggunaan: mengambil mahasiswa yang lulus, transaksi di atas nominal tertentu, atau data dengan status tertentu.

```
val angka2 = sc.parallelize(1 to 10)

val genap = angka2.filter(x => x % 2 == 0)
```

```
genap.collect()
```

Hasil:

```
Array(2, 4, 6, 8, 10)
```

12. Transformation: flatMap()

flatMap(): digunakan ketika satu elemen input dapat menghasilkan nol, satu, atau beberapa elemen output, lalu seluruh hasil tersebut diratakan menjadi satu RDD.

Ilustrasi sebelum → sesudah:

["Hadoop Spark", "Big Data"] → flatMap(split) → ["Hadoop", "Spark", "Big", "Data"]

Perbedaan dengan map(): map() mempertahankan satu hasil untuk setiap input, sedangkan flatMap() dapat menghasilkan banyak hasil dari satu input.

Contoh penggunaan: memecah kalimat menjadi kata, memecah daftar item, atau mengekstrak beberapa nilai dari satu record.

```
val kalimat = sc.parallelize(Seq("Hadoop Spark", "Big Data"))

val kata = kalimat.flatMap(line => line.split(" "))

kata.collect()
```

Hasil:

```
Array(Hadoop, Spark, Big, Data)
```

map() mengubah elemen; flatMap() dapat menghasilkan beberapa elemen dari satu input dan meratakannya.

13. Action: count()

count(): digunakan untuk mengetahui jumlah elemen dalam RDD. Karena count() adalah action, operasi ini memicu eksekusi transformation yang diperlukan.

Ilustrasi: [A, B, C, D] → count() → 4

Contoh penggunaan: menghitung jumlah baris, jumlah kata, atau jumlah record yang dihasilkan setelah filtering.

```
kata.count()
```

count() menghasilkan jumlah elemen RDD dan memicu eksekusi transformation yang diperlukan.

14. Action: first()

first(): digunakan untuk mengambil satu elemen pertama dari RDD. Operasi ini berguna untuk pemeriksaan cepat tanpa mengambil seluruh data.

Ilustrasi: ["Spark", "Hadoop", "HDFS"] → first() → "Spark"

Catatan: first() bukan untuk mengambil seluruh isi dataset. Untuk melihat seluruh data kecil gunakan collect().

```
kata.first()
```

first() mengambil elemen pertama.

15. Action: reduce()

reduce(): digunakan untuk menggabungkan elemen-elemen RDD menjadi satu nilai menggunakan fungsi penggabungan.

Ilustrasi: [1, 2, 3, 4, 5] → reduce(+) → 15

Contoh penggunaan: menjumlahkan nilai, mencari hasil agregasi sederhana, atau operasi penggabungan lain yang memenuhi aturan reduce.

Catatan: reduce() menghasilkan satu nilai akhir, sedangkan reduceByKey() menghasilkan agregasi untuk setiap key.

```
val angka3 = sc.parallelize(Seq(1,2,3,4,5))

val total = angka3.reduce((a,b) => a + b)

total
```

Hasil: 15.

16. WordCount dengan RDD

pipeline WordCount: digunakan untuk menghitung frekuensi kemunculan setiap kata pada kumpulan teks. Operasi flatMap(), map(), dan reduceByKey() menunjukkan bagaimana RDD dapat digunakan untuk membangun proses analisis data secara bertahap.

```
val teks = sc.parallelize(Seq("spark hadoop", "spark big data", "hadoop data"))

val words = teks.flatMap(line => line.split(" "))

val pairs = words.map(word => (word, 1))

val counts = pairs.reduceByKey((a,b) => a + b)

counts.collect()
```

Alurnya:

```
kalimat → flatMap() → kata → map() → (kata,1) → reduceByKey() → (kata,jumlah)
```

Hasil secara konsep:

```
(spark,2)

(hadoop,2)

(data,2)

(big,1)
```

Catatan: Urutan tampilan hasil dapat berbeda. Yang penting jumlah setiap kata benar.

17. Mengapa reduceByKey()?

GUNA reduceByKey(): digunakan untuk melakukan agregasi pada data berbentuk pasangan (key, value). Semua value yang mempunyai key sama digabungkan.

Ilustrasi:

[(spark,1), (hadoop,1), (spark,1), (spark,1)] → reduceByKey(+) → [(spark,3), (hadoop,1)]

Contoh penggunaan: WordCount, total transaksi per pelanggan, total penjualan per produk, atau jumlah data per kategori.

Mengapa bukan reduce()? Karena reduce() menggabungkan seluruh elemen, sedangkan reduceByKey() menggabungkan elemen berdasarkan key.

reduceByKey() digunakan pada pasangan key-value. Semua value dengan key yang sama digabungkan.

```
("spark",1)

("spark",1)

("hadoop",1)

       ↓ reduceByKey()

("spark",2)

("hadoop",1)
```

18. Membaca File dari HDFS

textFile(): digunakan untuk membuat RDD dari file teks, termasuk file yang tersimpan di HDFS. Dengan demikian Spark dapat memproses data yang sebelumnya disimpan oleh Hadoop/HDFS.

Ilustrasi: HDFS → textFile() → RDD → Partition → Task → Executor

Ini menunjukkan hubungan praktikum: HDFS berperan sebagai penyimpanan data, sedangkan Spark menggunakan RDD untuk memproses data tersebut.

Pastikan input.txt ada di HDFS. Dari CMD Windows:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/hdfs/input
```

Jika file terlihat, kembali ke spark-shell. Jika sudah keluar, masuk lagi:

```
/opt/spark/bin/spark-shell --master spark://spark-master:7077
```

Baca file:

```
val file = sc.textFile("hdfs://namenode:9000/praktikum/hdfs/input/input.txt")

file.count()

file.collect()
```

Catatan: Jika URI HDFS pada cluster Anda berbeda, gunakan nilai fs.defaultFS dari konfigurasi Hadoop yang digunakan pada praktikum sebelumnya.

19. WordCount dari HDFS

rangkaian operasi ini: flatMap() memecah baris menjadi kata, map() mengubah setiap kata menjadi pasangan (kata,1), dan reduceByKey() menjumlahkan kemunculan kata yang sama.

Ilustrasi lengkap:

"spark hadoop" → [spark, hadoop] → [(spark,1), (hadoop,1)] → agregasi → (spark,1), (hadoop,1)

Operasi ini merupakan contoh bagaimana beberapa transformation dapat dirangkai menjadi pipeline pemrosesan data.

```
val wordsHDFS = file.flatMap(line => line.split("\s+"))

val pairsHDFS = wordsHDFS.map(word => (word, 1))

val countsHDFS = pairsHDFS.reduceByKey((a,b) => a + b)

countsHDFS.collect()
```

Sekarang sumber data berasal dari HDFS, bukan Seq yang ditulis langsung di program.

20. Mengamati Lazy Evaluation

```
val hasil = file.filter(line => line.nonEmpty).map(line => line.toUpperCase)
```

Pada titik ini kita baru membuat transformation.

Jalankan action:

```
hasil.count()
```

Barulah Spark menjalankan pekerjaan yang diperlukan.

```
textFile() → filter() → map() → count()
```

21. Melihat Job, Stage, Task, Executor

Saat Spark Application aktif, buka:

```
http://localhost:4040
```

Amati Jobs, Stages, Tasks, dan Executors.

```
Action → Job → Stage → Task → Executor
```

Catatan: Ini adalah gambaran konseptual. Satu job dapat memiliki lebih dari satu stage, dan setiap stage dapat memiliki banyak task.

22. Cache dan In-Memory Processing

cache(): digunakan ketika sebuah RDD akan digunakan kembali beberapa kali. Spark dapat mempertahankan data hasil RDD sesuai mekanisme persistence yang digunakan sehingga perhitungan ulang dapat dikurangi pada penggunaan berikutnya.

Ilustrasi: HDFS → transformation → RDD hasil → cache() → digunakan untuk beberapa action. Tanpa cache, Spark dapat perlu menghitung ulang lineage ketika RDD tersebut dibutuhkan lagi.

cache() bukan berarti seluruh dataset harus selalu berada di RAM. Jika memori tidak cukup, Spark dapat menggunakan mekanisme penyimpanan/persistence yang sesuai.

Jika RDD akan digunakan berkali-kali, kita dapat meminta Spark mempertahankannya dengan cache().

```
val dataCache = file.filter(line => line.nonEmpty).cache()

dataCache.count()

dataCache.count()
```

Penggunaan berikutnya dapat memanfaatkan data yang telah dipertahankan sesuai resource yang tersedia.

Catatan: cache() tidak berarti seluruh dataset pasti masuk RAM. Jika memory tidak cukup, mekanisme persistence Spark dapat menggunakan storage lain sesuai kebutuhan.

23. Hubungan HDFS, RDD, Partition, dan RAM

```
HDFS

  ↓ membaca data

RDD

  ↓

RDD Partition

  ↓

Task

  ↓

Executor

  ↓

RAM / storage lokal sesuai kebutuhan
```

Data mentah tetap berada di HDFS. Data yang sedang diproses dapat berada pada resource executor. Jika RDD di-cache, Spark berusaha mempertahankannya untuk penggunaan berikutnya.

24. Perbandingan HDFS Block dan RDD Partition

Aspek	HDFS Block	RDD Partition	RDD

Tujuan	Penyimpanan	Pemrosesan	Representasi dataset
Pengelola	HDFS	Spark	Spark
Lokasi	DataNode	Diproses Executor	Terdistribusi
Unit	Block	Partition	Terdiri dari partition

25. Tugas Praktikum

1. Buat RDD berisi angka 1 sampai 20 dan tentukan jumlah partition.
2. Buat RDD dengan 4 partition dan amati dengan glom().
3. Gunakan map() untuk mengalikan setiap angka dengan 5.
4. Gunakan filter() untuk mengambil bilangan genap.
5. Gunakan flatMap() untuk memecah beberapa kalimat menjadi kata.
6. Gunakan reduce() untuk menghitung total angka.
7. Buat WordCount dari Seq menggunakan flatMap(), map(), dan reduceByKey().
8. Baca input.txt dari HDFS menggunakan textFile().
9. Lakukan WordCount terhadap input.txt dari HDFS.
10. Gunakan cache() pada RDD dan jalankan dua action.

26. Troubleshooting

Masalah	Tindakan awal
spark-shell tidak terhubung	Periksa docker ps dan spark://spark-master:7077.
File HDFS tidak ditemukan	Jalankan hdfs dfs -ls /praktikum/hdfs/input.
URI HDFS salah	Periksa fs.defaultFS pada cluster Hadoop.
Spark UI 4040 tidak terlihat	Pastikan Spark Application masih aktif.
collect() terlalu banyak output	Gunakan collect() hanya untuk data kecil; gunakan count() untuk data besar.
WordCount salah	Periksa split() dan gunakan split("\\s+") untuk whitespace.